

Описание алгоритмов и этапы разработки параллельных алгоритмов

Понятие алгоритма

«Алгоритм — это конечный набор правил, который определяет последовательность операций для решения конкретного множества задач и обладает пятью важными чертами: **конечность, определённость, ввод, вывод, эффективность**».

Дональд Кнут



Описание алгоритма

Запись на естественном языке:

«Макароны положить в кипящую воду (не менее 1 литра воды на 100 г. продукта), помешивая, варить до готовности».

Что означает «готовность»?

Вербальная запись алгоритма часто содержит неоднозначности, которые разными исполнителями могут трактоваться по-разному.

Запись формулой или на алгоритмическом языке :

$$S := a[1] + a[2] + a[3];$$

Каков порядок выполнения операций?

$S := (a[1] + a[2]) + a[3]$ или $S := a[1] + (a[2] + a[3])$?

Крокодила измеряем от головы к хвосту или от хвоста к голове?

Описание алгоритма

Запись формулой или на алгоритмическом языке

Пусть дробная часть числа может хранить только 4 десятичных разряда.

$$a[1] = 0.1024 \times 10^4, \quad a[2] = -0.1023 \times 10^4, \quad a[3] = 0.6 \times 10^0.$$

Первый порядок сложений:

$$a[1] + a[2] = 0.1024 \times 10^4 - 0.1023 \times 10^4 = 0.0001 \times 10^4 = 0.1 \times 10^1$$

$$(a[1] + a[2]) + a[3] = 0.1 \times 10^1 + 0.6 \times 10^0 = 1 \times 10^0 + 0.6 \times 10^0 = 1.6 \times 10^0 = 0.16 \times 10^1$$

Второй порядок сложений:

$$a[2] + a[3] = -0.1023 \times 10^4 + 0.6 \times 10^0 = -1023 \times 10^0 + 0.6 \times 10^0 = -1022.4 \times 10^0 =$$

$$-0.10224 \times 10^4 = -0.1022 \times 10^4$$

$$a[1] + (a[2] + a[3]) = 0.1024 \times 10^4 - 0.1022 \times 10^4 = 0.0002 \times 10^4 = \mathbf{0.2 \times 10^1}$$

Крокодил длиной **1.6** метра от головы к хвосту и длиной **2** метра от хвоста к голове!

Описание алгоритма

Граф алгоритма

Функционирование любой вычислительной системы сводится к выполнению двух видов работы: обработке информации и ее вводу-выводу.

Любой алгоритм, реализованный на компьютере, должен осуществлять некоторые действия над исходной информацией, задавая частичный порядок их выполнения.

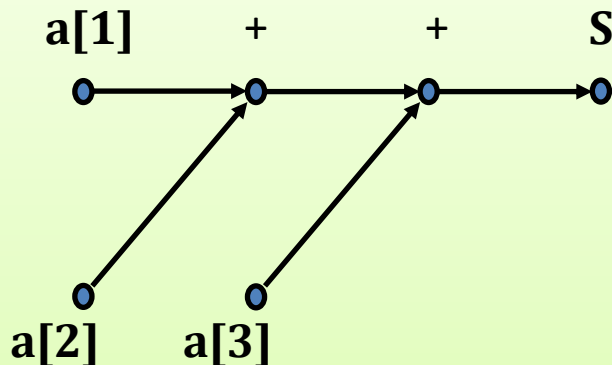
Результаты, полученные при исполнении одних операций, могут служить исходными данными для исполнения других операций.

Описание алгоритма

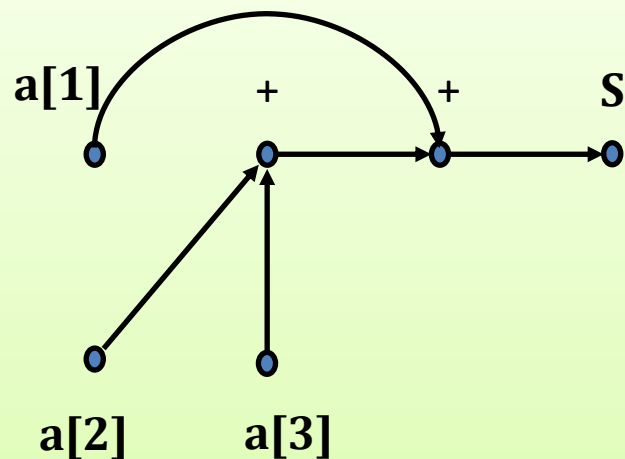
Граф алгоритма

Каждой операции в алгоритме сопоставим вершину графа.

Для наглядности отдельными вершинами отразим операции ввода и вывода. Если результат одной операции используется как данные при выполнении другой операции, то свяжем эти вершины направленным ребром.



$$S = (a[1] + a[2]) + a[3]$$

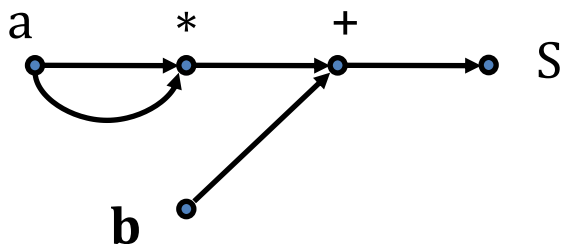


$$S = a[1] + (a[2] + a[3])$$

Описание алгоритма

Граф алгоритма. Свойства

1. Граф является ациклическим. В программах реализуются только явные вычисления.
2. Граф может быть мультиграфом.



$$S = (a*a) + b$$

3. Граф является параметрическим. Он всегда зависит от параметров масштаба задачи, определяющих общее количество вершин в графе.

Описание алгоритма

Граф алгоритма

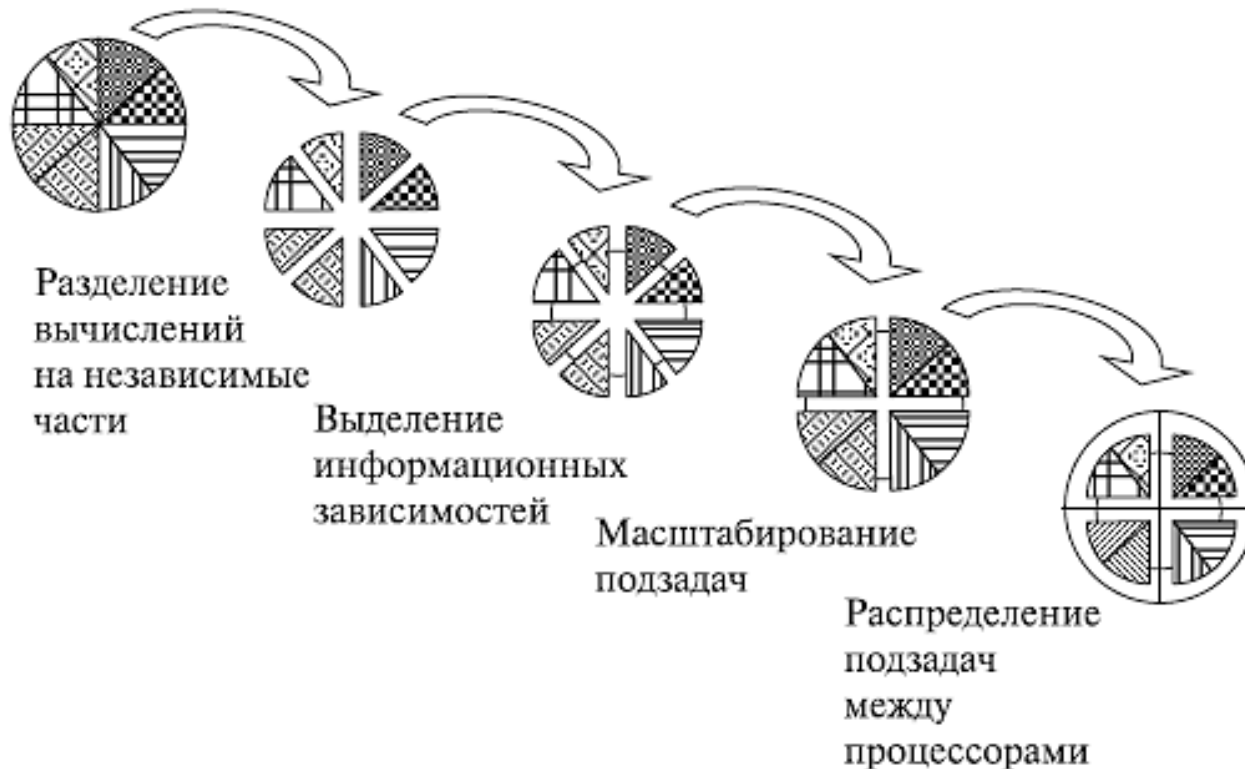
Функционирование любой вычислительной системы сводится к выполнению двух видов работы: обработке информации и ее вводу-выводу.

Любой алгоритм, реализованный на компьютере, должен осуществлять некоторые действия над исходной информацией, задавая частичный порядок их выполнения.

Результаты, полученные при исполнении одних операций, могут служить исходными данными для исполнения других операций.

Этапы разработки параллельных алгоритмов

1. Разделение вычислений на независимые части
2. Выделение информационных зависимостей
3. Масштабирование набора подзадач
4. Распределение подзадач между процессорами

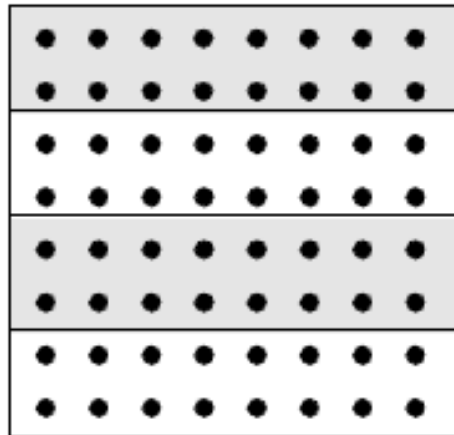


Этапы разработки параллельных алгоритмов (выполняемые действия)

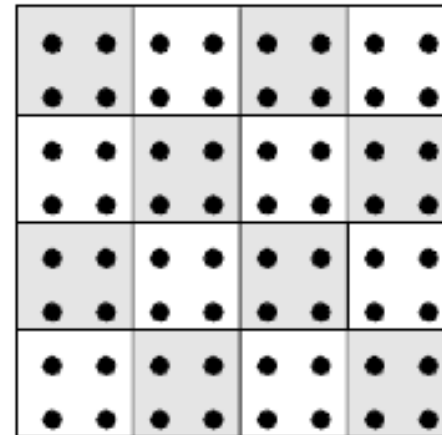
1. Выполнить анализ имеющихся вычислительных схем и осуществить их разделение (декомпозицию) на части (подзадачи), которые могут быть реализованы в значительной степени независимо друг от друга;
2. Выделить для сформированного набора подзадач информационные взаимодействия, которые должны осуществляться в ходе решения исходной поставленной задачи;
3. Определить необходимую (или доступную) для решения задачи вычислительную систему;
4. Выполнить распределение имеющего набора подзадач между процессорами системы.

Разделение вычислений на независимые части

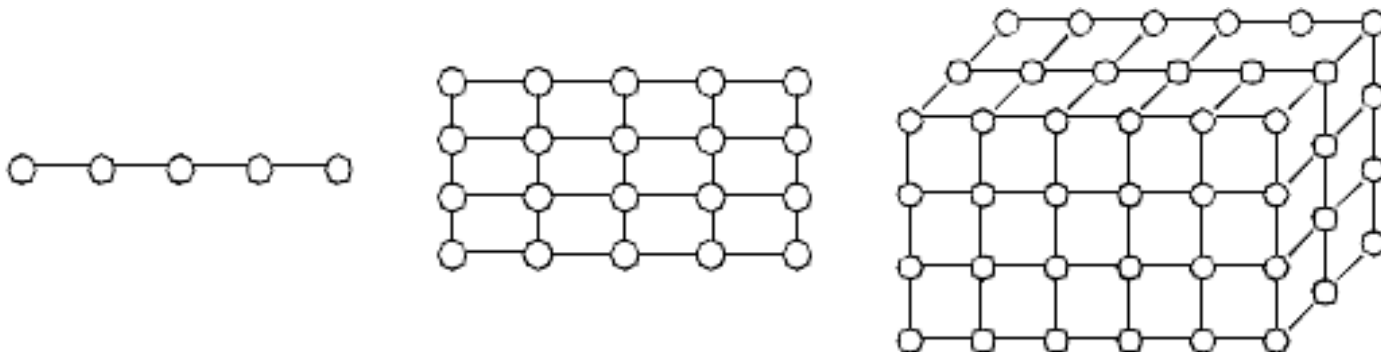
Типы вычислительных схем



ленточная схема



блочная схема



Регулярные одно-, дву- и трехмерные структуры
базовых подзадач после декомпозиции данных

Выделение информационных зависимостей

При проведении анализа информационных зависимостей между подзадачами следует различать:

локальные и *глобальные* схемы передачи данных – для локальных схем в каждый момент времени выполняются только между небольшим числом подзадач, для глобальных в процессе коммуникации принимают участие все подзадачи;

структурные и *произвольные* способы взаимодействия – для структурных способов организация взаимодействий приводит к формированию некоторых стандартных схем коммуникации (например, в виде кольца, прямоугольной решетки и т. д.), для произвольных структур взаимодействия схема выполняемых операций передач данных не носит характера однородности;

статические или *динамические* схемы передачи данных – для статических схем моменты и участники взаимодействия фиксируются на этапах проектирования и разработки, для динамического варианта взаимодействия структура операции передачи данных определяется в ходе выполняемых вычислений;

синхронные и *асинхронные* способы взаимодействия – для синхронных способов операции передачи выполняются только при готовности всех участников взаимодействия и завершаются только после полного окончания всех действий, при асинхронном выполнении операций участники взаимодействия могут не дожидаться полного завершения действий по передаче данных.

Масштабирование набора подзадач

Масштабирование разработанной вычислительной схемы параллельных вычислений проводится в случае, если количество имеющихся подзадач отличается от числа планируемых к использованию процессоров.

Для сокращения количества подзадач необходимо выполнить укрупнение (агрегацию) вычислений.

Применяемые правила: определяемые подзадачи, как и ранее, должны иметь одинаковую вычислительную сложность, а объем и интенсивность информационных взаимодействий между подзадачами должны оставаться на минимально возможном уровне. Как результат, первыми претендентами на объединение являются подзадачи с высокой степенью информационной взаимозависимости.

При недостаточном количестве имеющихся подзадач для загрузки всех доступных к использованию процессоров необходимо выполнить детализацию (декомпозицию) вычислений. Как правило, проведение подобной декомпозиции не вызывает каких-либо затруднений, если для базовых задач методы параллельных вычислений являются известными.

Выполнение этапа масштабирования вычислений должно свестись, в конечном итоге, к разработке правил агрегации и декомпозиции подзадач, которые должны параметрически зависеть от числа процессоров, применяемых для вычислений.

Распределение подзадач между процессорами (потоками)

Управление распределением нагрузки для процессоров возможно только для вычислительных систем с распределенной памятью.

Для мультипроцессоров (систем с общей памятью) распределение нагрузки обычно выполняется операционной системой автоматически.

Этап распределения подзадач между процессорами является избыточным, если количество подзадач совпадает с числом имеющихся процессоров, а топология сети передачи данных вычислительной системы представляет собой полный граф (т. е. все процессоры связаны между собой прямыми линиями связи).